# USBD35: Register a new facility - Sprint 3

Este documento descreve a implementação, configuração de dados e testes realizados para a **USBD35**.

## 1. Descrição da User Story

- As a Planner, I want to register a new facility

**Critérios de Aceitação e Regras de Negócio:**

- A facility é identificada por um ID (número) e um nome.
- O sistema deve validar se o ID não é NULL e se o nome não é NULL ou vazio.
- O sistema deve impedir a duplicação de IDs (chave primária).
- O sistema deve impedir a duplicação de nomes, ignorando diferenças de maiúsculas/minúsculas e espaços extra (normalização).
- A implementação deve ser feita através de uma função PL/SQL.

---

## 2. Explicação do Script (Função PL/SQL)

A lógica central da **USBD35** foi encapsulada na função PL/SQL `fn_register_facility`. Esta abordagem permite centralizar as regras de negócio e garantir a integridade dos dados, independentemente de onde a inserção é solicitada.

**Detalhes da Implementação:**

1. **Assinatura da Função:** A função aceita dois parâmetros: `p_facility_id` (ID) e `p_name` (Nome). Retorna o ID inserido ( `NUMBER` ).
2. **Validação de Dados Obrigatórios:** Verifica se o ID é NULL (lança erro -20001) ou se o nome é NULL/vazio (lança erro -20002).
3. **Verificação de Duplicados (ID):** Consulta a tabela `Facility` para verificar se o ID já existe. Se sim, lança erro -20003.
4. **Normalização de Texto:** Para garantir que a verificação de duplicados é robusta, o nome de entrada é normalizado:
    - Espaços nas extremidades removidos ( `TRIM` ).
    - Múltiplos espaços internos substituídos por um único espaço ( `REGEXP_REPLACE` ).
    - Convertido para maiúsculas para comparação ( `UPPER` ).
5. **Verificação de Duplicados (Nome):** Consulta a tabela `Facility` comparando o nome normalizado. Se já existir, lança erro -20004.
6. **Inserção Segura:** Executa o `INSERT` usando o ID fornecido e o nome limpo, seguido de `COMMIT` .
7. **Transação em Erro:** Em caso de exceção, executa `ROLLBACK` e relança o erro via `RAISE` .

### Código: `USBD35-script.sql`

```
CREATE OR REPLACE FUNCTION fn_register_facility(
    p_facility_id IN NUMBER,
    p_name        IN VARCHAR2
) RETURN NUMBER IS
    v_clean_name     VARCHAR2(100);
    v_normalized_in  VARCHAR2(100);
    v_count          NUMBER;
BEGIN
    -- 1. Validate Input: Check for NULLs
    IF p_facility_id IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001, 'Facility ID cannot be null.');
END IF;

    IF p_name IS NULL OR TRIM(p_name) IS NULL THEN
        RAISE_APPLICATION_ERROR(-20002, 'Facility name cannot be null or empty.');
END IF;

    -- 2. Check if Facility ID already exists
SELECT COUNT(*) INTO v_count FROM Facility WHERE id = p_facility_id;
IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20003, 'Facility ID ' || p_facility_id || ' already exists.');
END IF;

    -- 3. Normalize Input: Trim and collapse multiple spaces
    v_clean_name := REGEXP_REPLACE(TRIM(p_name), '\s+', ' ');
    v_normalized_in := UPPER(v_clean_name);

    -- 4. Case and Space Insensitive Duplicate Check (Name)
SELECT COUNT(*)
INTO v_count
FROM Facility
WHERE UPPER(REGEXP_REPLACE(TRIM(name), '\s+', ' ')) = v_normalized_in;

IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20004, 'Facility already exists (case/space insensitive check).');
END IF;

    -- 5. Insert new Facility using the provided ID
INSERT INTO Facility (id, name)
VALUES (p_facility_id, v_clean_name);

-- 6. TRANSAÇÃO: Sucess
COMMIT;
-- Return the ID that was just inserted
RETURN p_facility_id;

EXCEPTION
    WHEN OTHERS THEN
        -- 7. TRANSAÇÃO: Erro
        ROLLBACK;
        RAISE;
END;
/
```

# 3. Explicação de Cada Teste

Para validar a User Story USBD35, foram desenvolvidos 5 cenários de teste que cobrem inserções válidas, violações de integridade e regras de normalização de dados.

### Teste 1: Register Valid Facility

**Objetivo:** Verificar se é possível registrar uma nova facility válida que ainda não existe no sistema.
**Descrição:** Realiza limpeza preventiva deletando registros conflitantes, executa a função com ID 901 e nome 'Coimbra Logistics Hub', e verifica o retorno.
**Resultado Esperado:** A função deve retornar o ID (901) e o registro deve ser persistido.

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
    v_test_id NUMBER := 901;
    v_name VARCHAR2(100) := 'Coimbra Logistics Hub';
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 1: Register Valid Facility ---');
    -- Limpeza preventiva
DELETE FROM Facility WHERE id = v_test_id OR UPPER(name) = UPPER(v_name);
COMMIT;

v_id := fn_register_facility(v_test_id, v_name);
    DBMS_OUTPUT.PUT_LINE('RESULT: SUCCESS. Returned ID: ' || v_id);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('RESULT: FAILURE. Error: ' || SQLERRM);
END;
/
```

## Teste 2: Normalization (Case & Spaces)

**Objetivo:** Verificar se o sistema normaliza nomes (casing e espaços) e impede duplicados.
**Descrição:** Realiza limpeza preventiva, insere uma facility base com ID 902 e nome 'Porto Test Hub', então tenta inserir uma variação normalizada com ID 903.
**Resultado Esperado:** A segunda inserção deve falhar com erro -20004 (duplicado).

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 2: Normalization (Case & Spaces) ---');
    -- Limpeza de IDs de teste
DELETE FROM Facility WHERE id IN (902, 903) OR UPPER(name) LIKE '%PORTO TEST HUB%';
COMMIT;

v_id := fn_register_facility(902, 'Porto Test Hub');

BEGIN
        v_id := fn_register_facility(903, '   PORTO   test   HUB   ');
        DBMS_OUTPUT.PUT_LINE('RESULT: FAILURE. Allowed duplicate.');
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20004 THEN
                DBMS_OUTPUT.PUT_LINE('RESULT: SUCCESS. Caught duplicate.');
ELSE RAISE;
END IF;
END;
END;
/
```

## Teste 3: Null Input Validation

**Objetivo:** Verificar se o sistema impede inserções com nome NULL.
**Descrição:** Tenta registrar com ID 904 e nome NULL.
**Resultado Esperado:** Deve lançar erro -20002.

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 3: Null Name Validation ---');
BEGIN
        v_id := fn_register_facility(904, NULL);
        DBMS_OUTPUT.PUT_LINE('RESULT: FAILURE.');
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20002 THEN
                DBMS_OUTPUT.PUT_LINE('RESULT: SUCESSO (NULL bloqueado).');
ELSE DBMS_OUTPUT.PUT_LINE('RESULT: FALHA (Erro inesperado): ' || SQLERRM);
END IF;
END;
END;
/
```

## Teste 4: Duplicate against USBD32 Seed Data

**Objetivo:** Verificar se o sistema impede duplicados com dados de seed existentes (ex: 'Valença').
**Descrição:** Tenta inserir variação normalizada de 'Valença' com ID 905, assumindo que o seed data está carregado.
**Resultado Esperado:** Deve falhar com erro -20004 se 'Valença' for detectado como duplicado.

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 4: Seed Data Duplicate ---');
    -- Testa contra 'Valença' (carregada na USBD32)
BEGIN
        v_id := fn_register_facility(905, '  valença  ');
        DBMS_OUTPUT.PUT_LINE('RESULT: FAILURE.');
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20004 THEN
                DBMS_OUTPUT.PUT_LINE('RESULT: SUCESSO (Duplicado de Valença detetado).');
ELSE DBMS_OUTPUT.PUT_LINE('RESULT: FALHA: ' || SQLERRM);
END IF;
END;
END;
/
```

## Teste 5: Empty String / Spaces Validation

**Objetivo:** Verificar se o sistema impede inserções com nome vazio ou apenas espaços.
**Descrição:** Tenta registrar com ID 906 e nome ' '.
**Resultado Esperado:** Deve lançar erro -20002.

```
 SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 5: Empty Spaces Validation ---');
BEGIN
        v_id := fn_register_facility(906, '      ');
        DBMS_OUTPUT.PUT_LINE('RESULT: FAILURE.');
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20002 THEN
                DBMS_OUTPUT.PUT_LINE('RESULT: SUCESSO (Espaços bloqueados).');
ELSE DBMS_OUTPUT.PUT_LINE('RESULT: FALHA: ' || SQLERRM);
END IF;
END;
END;
/
```